DEVOPS FINAL CAPSTONE

GameSense

A Steam-style game discovery app with a content-based recommender, shipped through CI/CD, Terraform on AWS EKS, GitOps, and observability.

Nishant Dasgupta

16 October 2026 · github.com/NDGCodes/GameSense

OVERVIEW

What I built

Game discovery web app

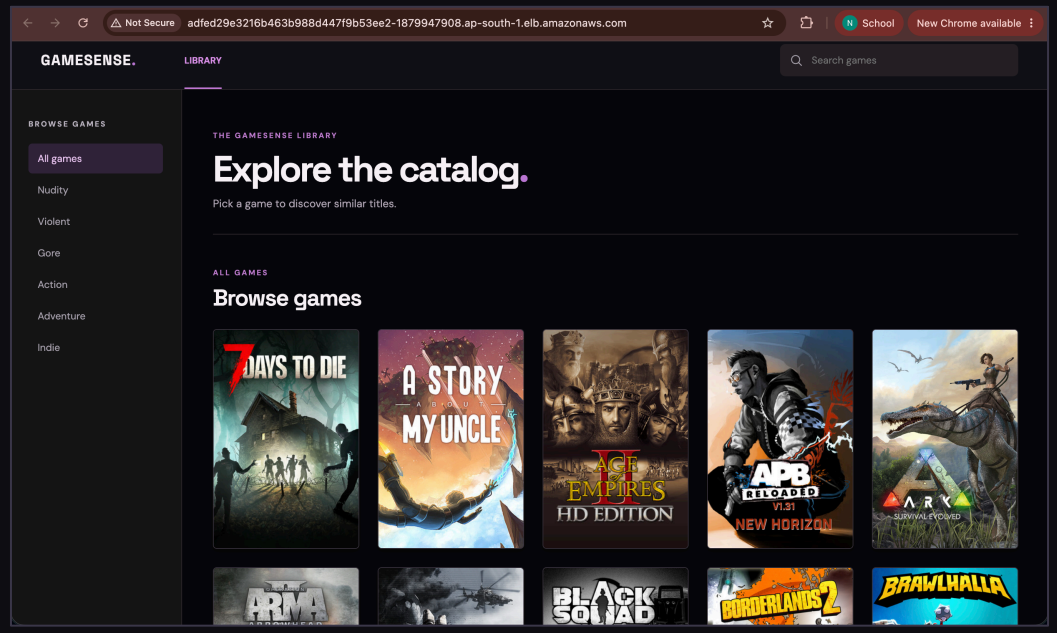
Browse and search a Steam catalog, filter by genre, open a game, and get similar-game recommendations with artwork.

Content-based recommender

TF-IDF over tags, genres, and descriptions, fitted on the Kaggle Steam Store Games dataset (27,075 games locally; a 100-game subset on EKS).

Production-style delivery

Tested, scanned, containerised, and deployed to AWS EKS by GitOps, with Prometheus and Grafana monitoring.



Live on EKS through the Traefik load balancer

LANGUAGES, TOOLS, PLATFORMS

Tech stack

Application

Python 3.11 · FastAPI · SQLAlchemy · Alembic

React 19 · Vite 8 · Nginx

PostgreSQL 16 (pg8000)

ML

TF-IDF + tag similarity in pure Python

Kaggle Steam Store Games (2019)

Fitted in a Kaggle notebook

Quality & security

pytest · npm build

Trivy (HIGH/CRITICAL gate)

helm lint · terraform validate

Delivery

Docker · Docker Compose

GitHub Actions · GHCR

Argo CD (GitOps)

Infrastructure

Terraform · AWS VPC · EKS 1.35

Helm · Traefik ingress

kubectl

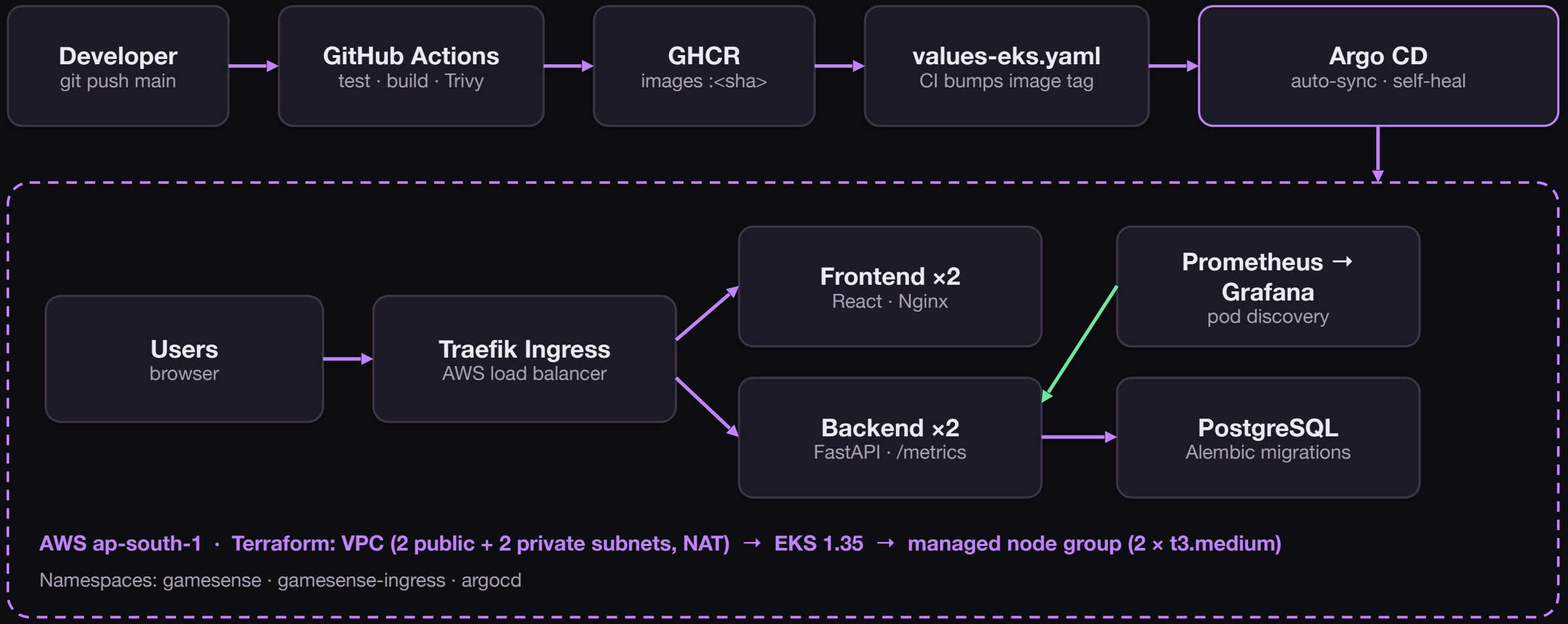
Observability

prometheus_client · Prometheus

Grafana dashboards

END-TO-END FLOW

Architecture



FRONTEND · BACKEND · DATABASE

Application (M1)

Frontend: React + Vite

Catalog grid, search, genre filters, detail view with similar games. Served by Nginx, which proxies /api to the backend.

Backend: FastAPI

Health and readiness checks, catalog, recommendations, and a saved-games list persisted in PostgreSQL.

Database: PostgreSQL + Alembic

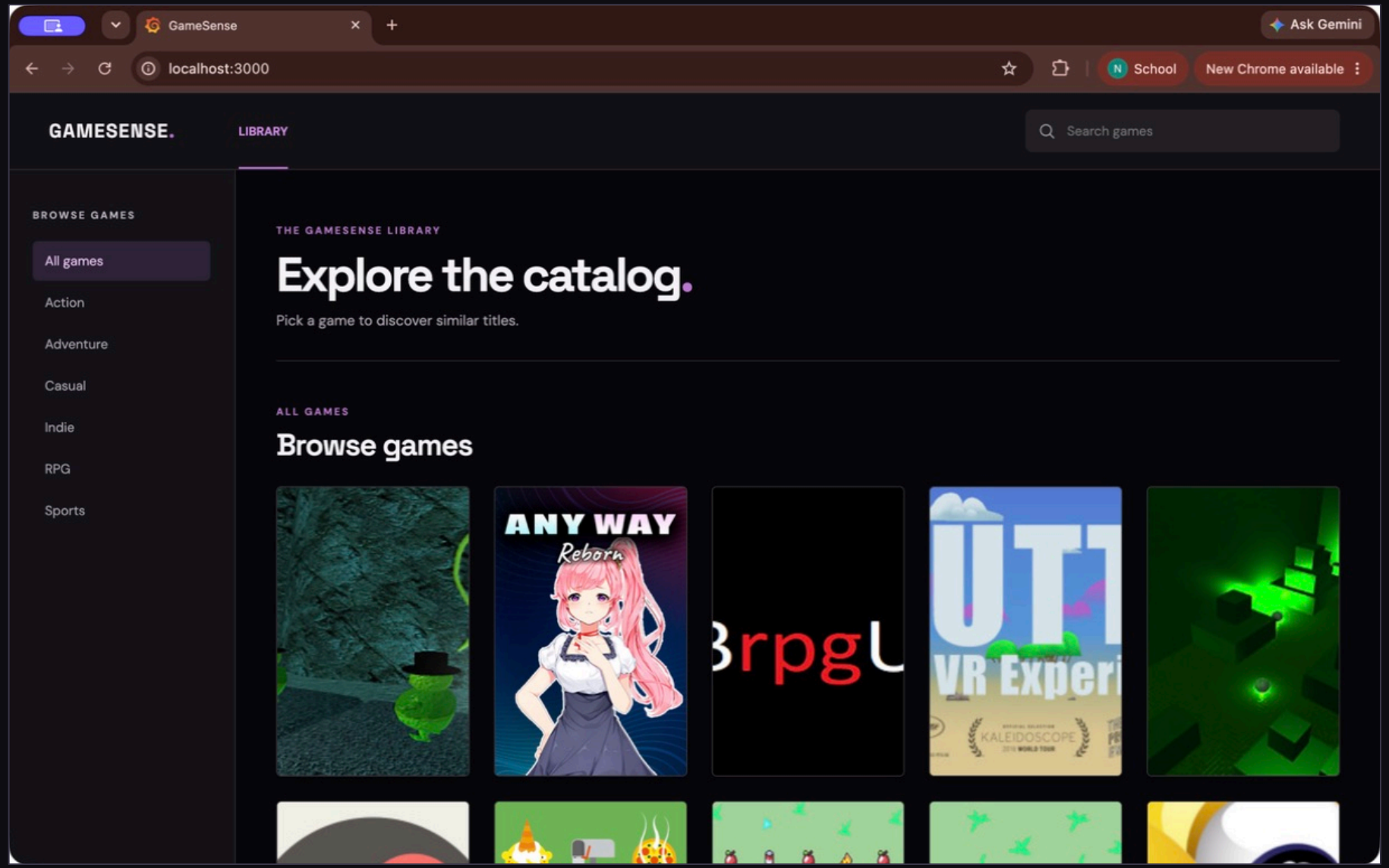
Three migrations: demo catalog, canonical catalog tables, saved-game list.

REST API

GET	/health · /ready · /metrics
GET	/api/games
POST	/api/recommendations
GET	/api/saved-games
PUT	/api/saved-games/{slug}
DELETE	/api/saved-games/{slug}

EVIDENCE

App running locally (M1 · M4)



React UI at http://localhost:3000 via docker compose, calling the FastAPI backend

QUALITY GATES

Testing, Git & containers (M2 · M3 · M4)

pytest: 59 tests passing

API, Alembic migrations, catalog schemas, recommender, ML data pipeline, and artwork. API tests run against a fresh temporary SQLite database built by the Alembic migrations, never the production database.

Git workflow

Feature branches and pull requests into main, meaningful commits, and a .gitignore covering secrets, state, data, and build output.

Docker

Backend runs as a non-root user (UID 999). Frontend is a multi-stage Node build into an unprivileged Nginx image (UID 101) on port 8080.

Docker Compose

PostgreSQL, backend, frontend, Prometheus, and Grafana, with health checks and a named database volume. App on :3000, Grafana on :3001.

EVIDENCE

pytest: all tests pass (M2) (1/2)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % python -m pytest -v backend/tests

============================= test session starts ==============================
platform darwin -- Python 3.10.13, pytest-8.3.4, pluggy-1.6.0 -- /private/tmp/gamesense-kaggle-run-venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/nishantdasgupta/Documents/ML_Ops/GameSense
plugins: anyio-4.15.1
collecting ... collected 59 items

backend/tests/test_api.py::test_alembic_migration_creates_labeled_demo_fixtures PASSED [  1%]
backend/tests/test_api.py::test_recommendations_come_from_demo_database_fixtures PASSED [  3%]
backend/tests/test_api.py::test_content_recommender_mode_uses_self_authored_sample_and_grounded_reasons PASSED [  5%]
backend/tests/test_api.py::test_content_recommender_fallback_metric_uses_bounded_reason PASSED [  6%]
backend/tests/test_api.py::test_content_recommender_loads_returned_bundle_for_catalog_and_recommendations PASSED [  8%]
backend/tests/test_api.py::test_readiness_checks_database_and_counts_catalog PASSED [ 10%]
backend/tests/test_api.py::test_saved_games_put_update_delete_persist_and_validate PASSED [ 11%]
backend/tests/test_api.py::test_saved_games_preserve_existing_database_data PASSED [ 13%]
backend/tests/test_api.py::test_concurrent_saved_game_puts_keep_one_row PASSED [ 15%]
backend/tests/test_catalog_foundation.py::test_sqlite_migrations_create_canonical_catalog_without_changing_demo_fixtures PASSED [
16%]
backend/tests/test_catalog_foundation.py::test_source_manifest_example_validates_as_catalog_source_schema PASSED [ 18%]
backend/tests/test_catalog_foundation.py::test_catalog_game_feature_and_artifact_schemas_validate_canonical_records PASSED [ 20%]
backend/tests/test_catalog_foundation.py::test_catalog_schemas_reject_invalid_identifiers PASSED [ 22%]
backend/tests/test_game_images.py::test_legacy_bundle_identity_provides_same_id_image_without_title_matching PASSED [ 23%]
backend/tests/test_game_images.py::test_dataset_image_metadata_takes_precedence_for_exact_game PASSED [ 25%]
backend/tests/test_game_images.py::test_duplicate_titles_keep_distinct_identity_bound_covers PASSED [ 27%]
backend/tests/test_game_images.py::test_unknown_or_inconsistent_identity_has_no_artwork[overrides0] PASSED [ 28%]
```

pytest -v: API, migration, and catalog tests

EVIDENCE

pytest: all tests pass (M2) (2/2)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % python -m pytest -v backend/tests

backend/tests/test_steam_store_bundle.py::test_similar_fallback_excludes_seed_in_both_content_paths[Crafting-expected_slugs1] PASSED
[ 96%]
backend/tests/test_steam_store_bundle.py::test_load_bundle_rejects_id_mapping_mismatch PASSED [ 98%]
backend/tests/test_steam_store_bundle.py::test_kaggle_notebook_exports_bundle_and_run_report PASSED [100%]

============================== warnings summary ===============================
../../../../../private/tmp/gamesense-kaggle-run-venv/lib/python3.10/site-packages/fastapi/testclient.py:1
  /private/tmp/gamesense-kaggle-run-venv/lib/python3.10/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using
`httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 59 passed, 1 warning in 9.21s ========================
```

Suite result: 59 passed

EVIDENCE

Commit history (M3)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % git log -12 --format="%h  %s"

a23561c  Refresh current evidence
ff4734a  Close local deployment rubric gaps
dff325f  UI & catalog-first recommendations
e20bced  Catalog artwork gaps with Steam capsule fallbacks
c643a49  Display Steam game artwork in recommendation cards
c63d4d0  Kaggle Steam catalog
19f430e  local evidence submission
51fbde1  no-mistakes(document): Consolidate submission evidence references and correct fixture guidance
b857a67  Add M9 local submission package
a403a4e  Local evidence freeze
643118f  no-mistakes(document): Consolidate M8 evidence references and clarify Python coverage
d2df85f  Document M8 local evidence freeze
```

Meaningful commit messages on main

EVIDENCE

docker compose up --build (M4) (1/2)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % docker compose -p gamesense-submission -f docker-compose.yml -f <evidence-
override> up --build -d --wait

 Container gamesense-submission-db-1 Started
 Container gamesense-submission-db-1 Waiting
 Container gamesense-submission-db-1 Healthy
 Container gamesense-submission-backend-1 Starting
 Container gamesense-submission-backend-1 Started
 Container gamesense-submission-backend-1 Waiting
 Container gamesense-submission-backend-1 Waiting
 Container gamesense-submission-backend-1 Healthy
 Container gamesense-submission-backend-1 Healthy
 Container gamesense-submission-frontend-1 Starting
 Container gamesense-submission-prometheus-1 Starting
 Container gamesense-submission-frontend-1 Started
 Container gamesense-submission-prometheus-1 Started
 Container gamesense-submission-grafana-1 Starting
 Container gamesense-submission-grafana-1 Started
 Container gamesense-submission-backend-1 Waiting
 Container gamesense-submission-frontend-1 Waiting
 Container gamesense-submission-prometheus-1 Waiting
 Container gamesense-submission-grafana-1 Waiting
 Container gamesense-submission-db-1 Waiting
 Container gamesense-submission-prometheus-1 Healthy
 Container gamesense-submission-db-1 Healthy
 Container gamesense-submission-backend-1 Healthy
 Container gamesense-submission-grafana-1 Healthy
 Container gamesense-submission-frontend-1 Healthy
```

All services start and report healthy

EVIDENCE

docker compose up --build (M4) (2/2)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % docker compose -p gamesense-submission ... ps; docker compose ... exec -T
<service> id

NAME                                IMAGE                                COMMAND                  SERVICE      CREATED              STATUS
PORTS
gamesense-submission-backend-1      gamesense-submission-backend         "sh -c 'alembic upgr…"   backend      About a minute ago   Up 53
seconds (healthy)       127.0.0.1:28000->8000/tcp
gamesense-submission-db-1           postgres:16-alpine                   "docker-entrypoint.s…"   db           About a minute ago   Up
About a minute (healthy)   5432/tcp
gamesense-submission-frontend-1     gamesense-submission-frontend        "/docker-entrypoint.…"   frontend     About a minute ago   Up 48
seconds (healthy)       127.0.0.1:3000->8080/tcp
gamesense-submission-grafana-1      grafana/grafana:11.2.0               "/run.sh"                grafana      About a minute ago   Up 47
seconds                 127.0.0.1:23000->3000/tcp
gamesense-submission-prometheus-1   prom/prometheus:v2.54.1              "/bin/prometheus --c…"   prometheus   About a minute ago   Up 48
seconds                 127.0.0.1:29090->9090/tcp

Backend: uid=999(gamesense) gid=999(gamesense) groups=999(gamesense)
Frontend: uid=101(nginx) gid=101(nginx) groups=101(nginx)
```

Containers run as non-root (backend UID 999, frontend UID 101)

GITHUB ACTIONS

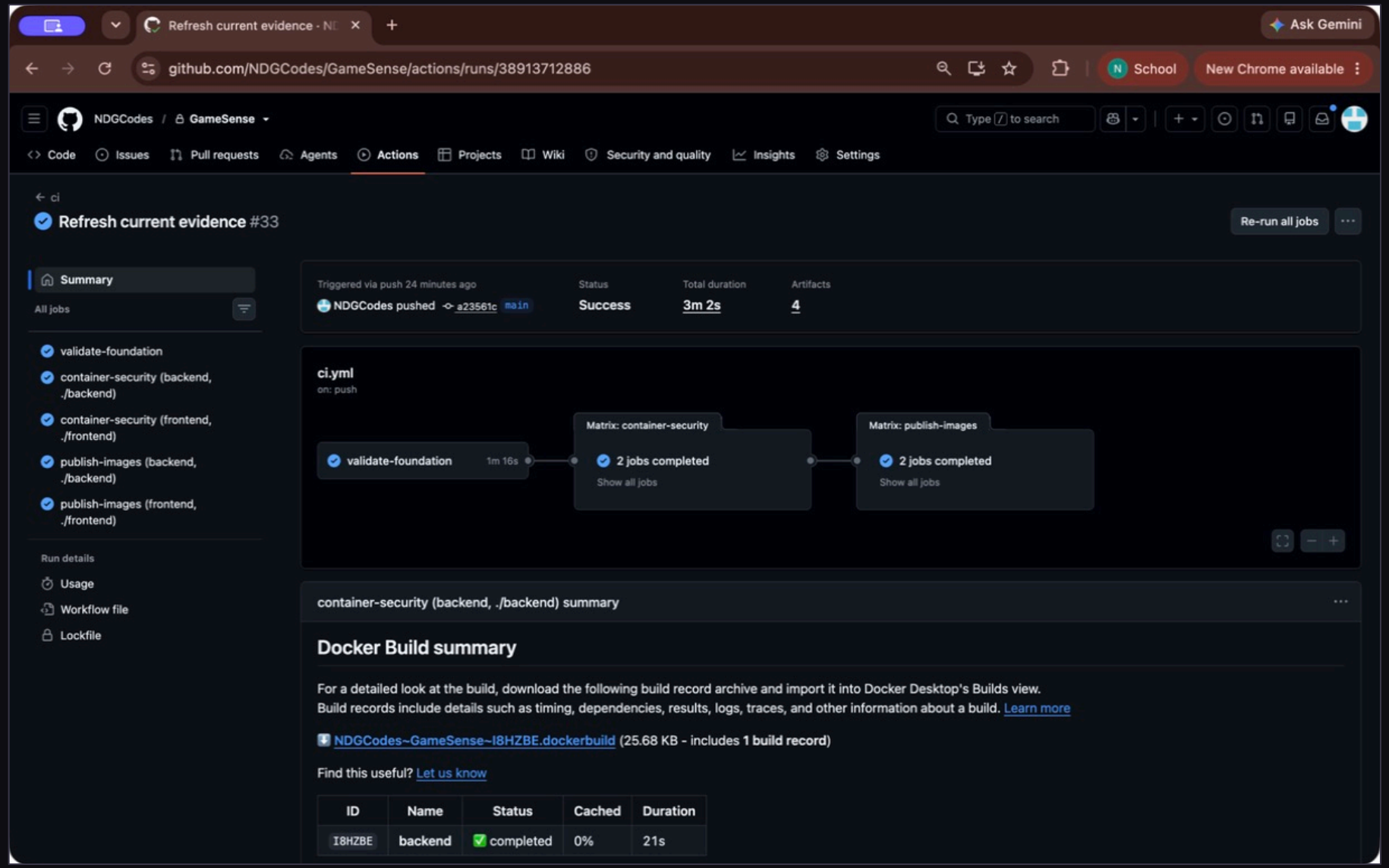
CI/CD pipeline (M5 · M6)



- Runs on every push and pull request. Images are published and deployed only from main.
- Trivy scans both images for OS and library CVEs; any fixable HIGH or CRITICAL finding fails the build before anything is pushed.
- The final job commits the new tag to Git as github-actions[bot]. Argo CD sees that commit and deploys it, so no deployment is run by hand.

EVIDENCE

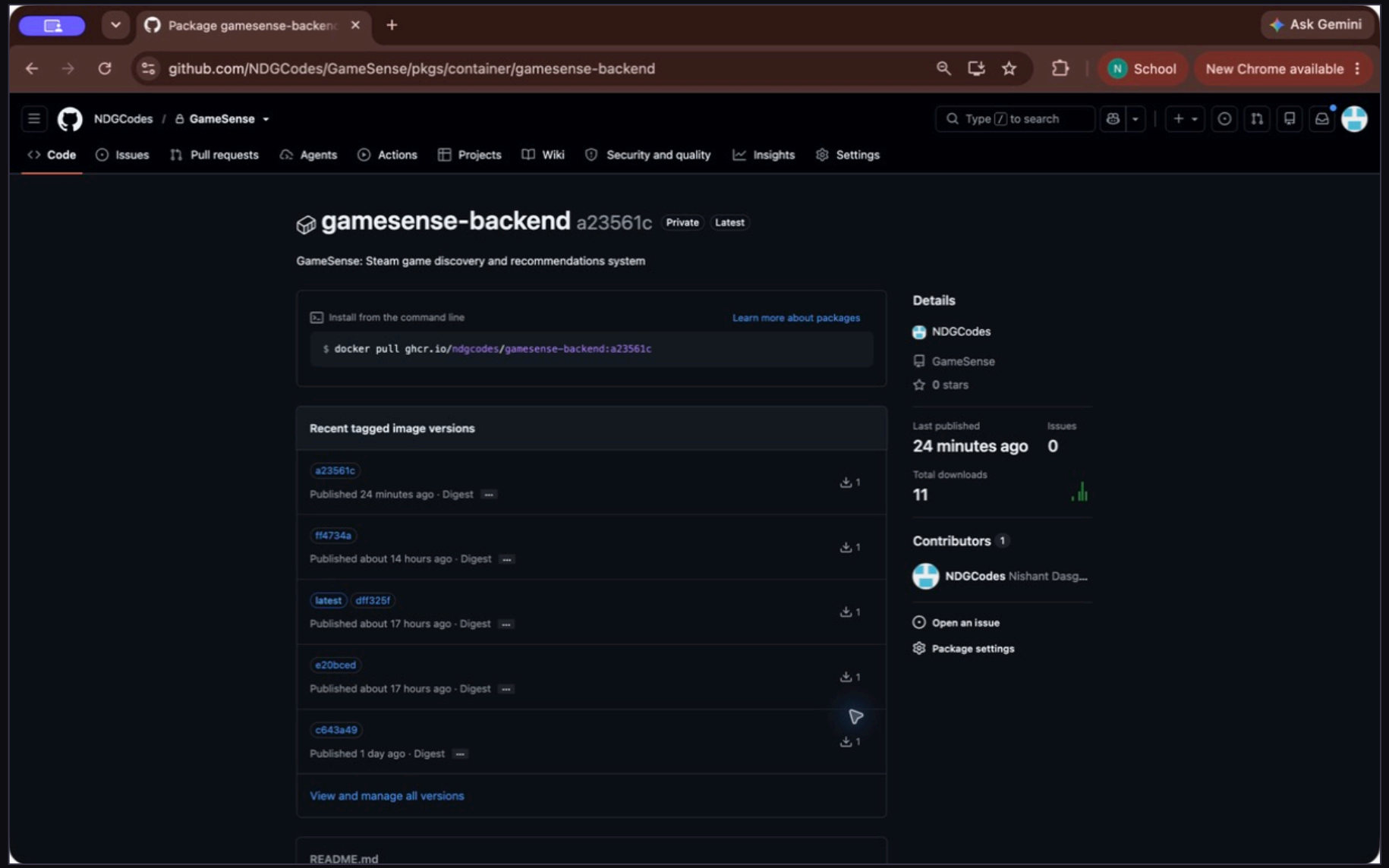
GitHub Actions: green pipeline (M5)



Push to main: validate → container-security → publish-images, all green

EVIDENCE

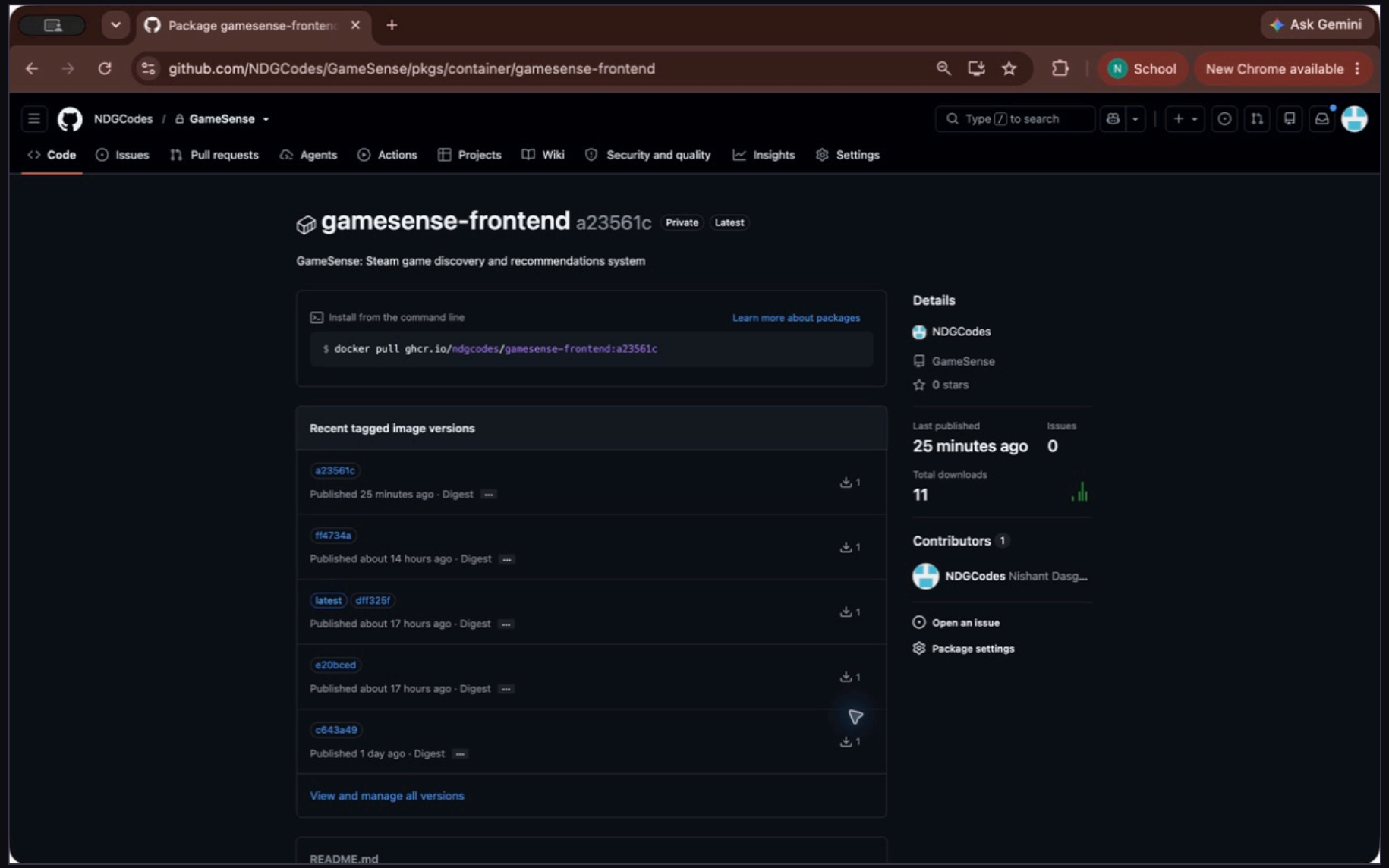
GHCR images tagged with the commit SHA (M5) (1/2)



ghcr.io/ndgcodes/gamesense-backend

EVIDENCE

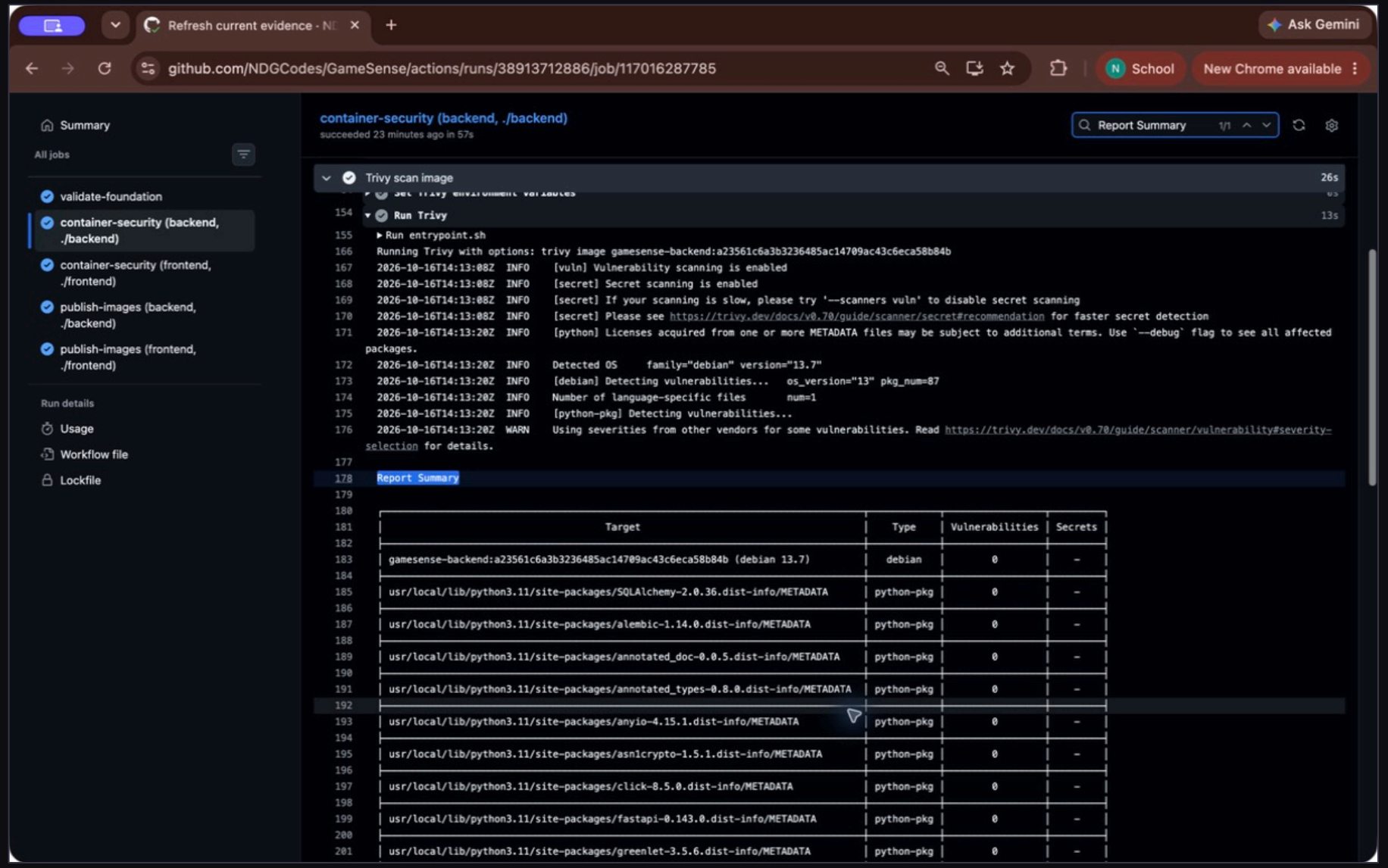
GHCR images tagged with the commit SHA (M5) (2/2)



ghcr.io/ndgcodes/gamesense-frontend

EVIDENCE

Trivy scan in CI (M6)



Backend image: 0 HIGH/CRITICAL vulnerabilities (gate passes; the build fails if any are found)

TERRAFORM ON AWS

Infrastructure as code (M7)

Network

VPC 10.42.0.0/16 across two AZs in ap-south-1 (Mumbai): 2 public and 2 private subnets, one NAT gateway, and the subnet tags EKS uses for load balancers.

Cluster

EKS 1.35 with a managed node group of 2 × t3.medium (AL2023). The cluster creator gets admin access.

Safety

allowed_account_ids guards against the wrong account. State and plans stay out of Git. Optional ECR repos are force-deleted on destroy.

Lifecycle

terraform init

terraform fmt -check · validate

terraform plan -out=<saved plan>

terraform apply <saved plan>

terraform destroy (after the demo)

EVIDENCE

terraform plan (M7)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % terraform plan -var-file=<private configuration> -out=<private plan> -no-color

Plan: 58 to add, 0 to change, 0 to destroy.


  with module.eks.aws_iam_role.this[0],
  on .terraform/modules/eks/main.tf line 402, in resource "aws_iam_role" "this":
 402: resource "aws_iam_role" "this" {

inline_policy is deprecated. Use the aws_iam_role_policy resource instead. If
Terraform should exclusively manage all inline policy associations (the
current behavior of this argument), use the aws_iam_role_policies_exclusive
resource as well.

─────────────────────────────────────────────────────────────────────────────

Saved the plan to: ../../submission-evidence/2026-10-16/private/mumbai.tfplan

To perform exactly these actions, run the following command to apply:
    terraform apply "../../submission-evidence/2026-10-16/private/mumbai.tfplan"
```

Plan: 58 to add, 0 to change, 0 to destroy

EVIDENCE

terraform apply (M7)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % terraform apply <saved plan>

module.vpc.aws_vpc.this[0]: Creation complete after 13s [id=vpc-0ee14bf138d7e9b63]
module.vpc.aws_subnet.public[1]: Creation complete after 1s [id=subnet-0a2e5264ff7d685c1]
module.vpc.aws_subnet.public[0]: Creation complete after 1s [id=subnet-0a57baeda07aef588]
module.vpc.aws_nat_gateway.this[0]: Creation complete after 1m34s [id=nat-011d42463b93b7ad8]
module.eks.aws_eks_cluster.this[0]: Creation complete after 8m8s [id=gamesense-capstone]
module.eks.module.eks_managed_node_group["gamesense_workers"].aws_eks_node_group.this[0]: Creation complete after 1m50s [id=gamesense-
capstone:gamesense_workers-20261016144547160700000011]

Apply complete! Resources: 58 added, 0 changed, 0 destroyed.

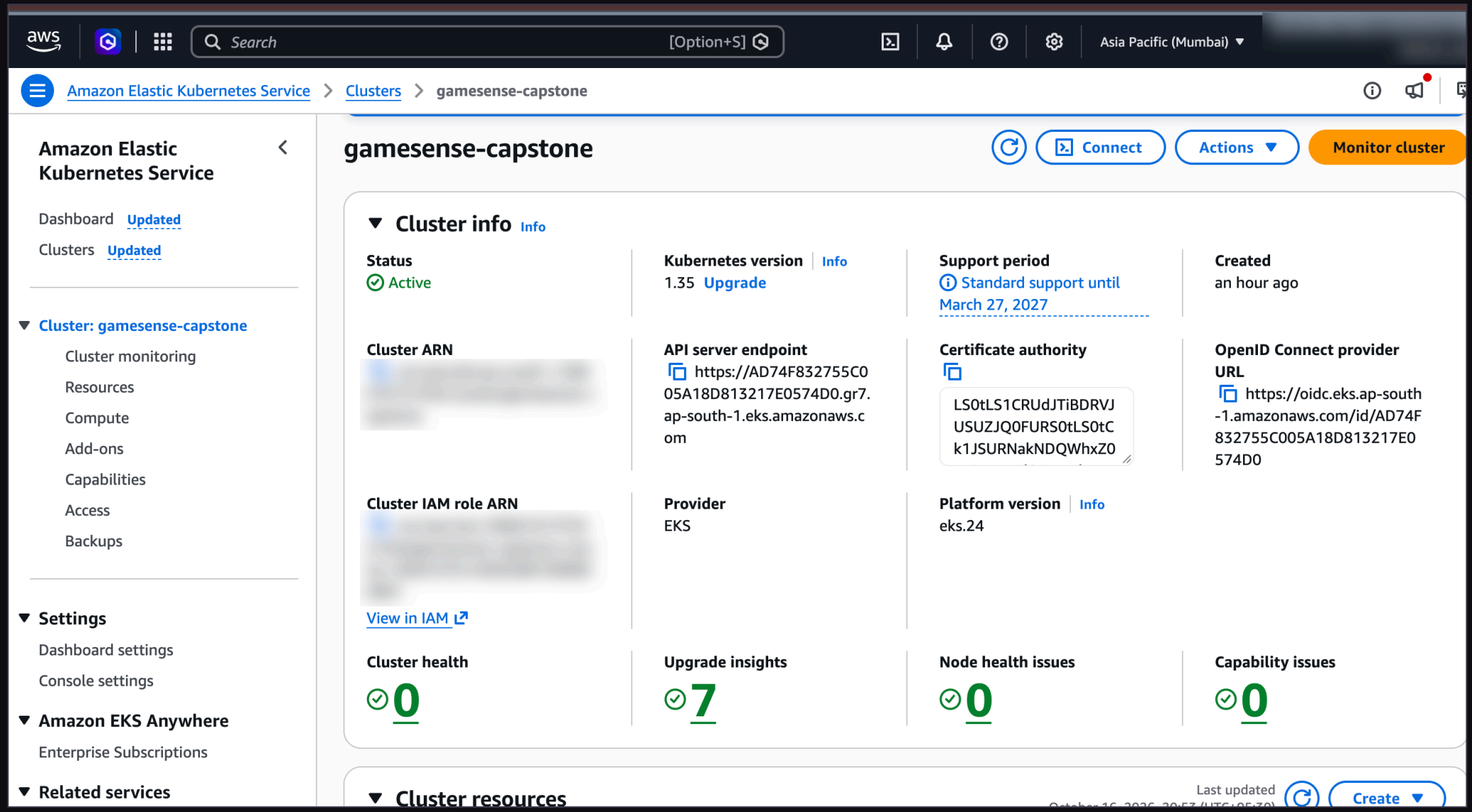
Outputs:

cluster_endpoint = "https://AD74F832755C005A18D813217E0574D0.gr7.ap-south-1.eks.amazonaws.com"
cluster_name = "gamesense-capstone"
configure_kubectl = "aws eks update-kubeconfig --region ap-south-1 --name gamesense-capstone"
private_subnets = [
  "subnet-037103c6a0a9b62fb",
  "subnet-08520e63f78d5abd2",
]
public_subnets = [
  "subnet-0a57baeda07aef588",
  "subnet-0a2e5264ff7d685c1",
]
vpc_id = "vpc-0ee14bf138d7e9b63"
```

VPC, 2 public subnets, NAT, EKS cluster, and node group created: 58 resources

EVIDENCE

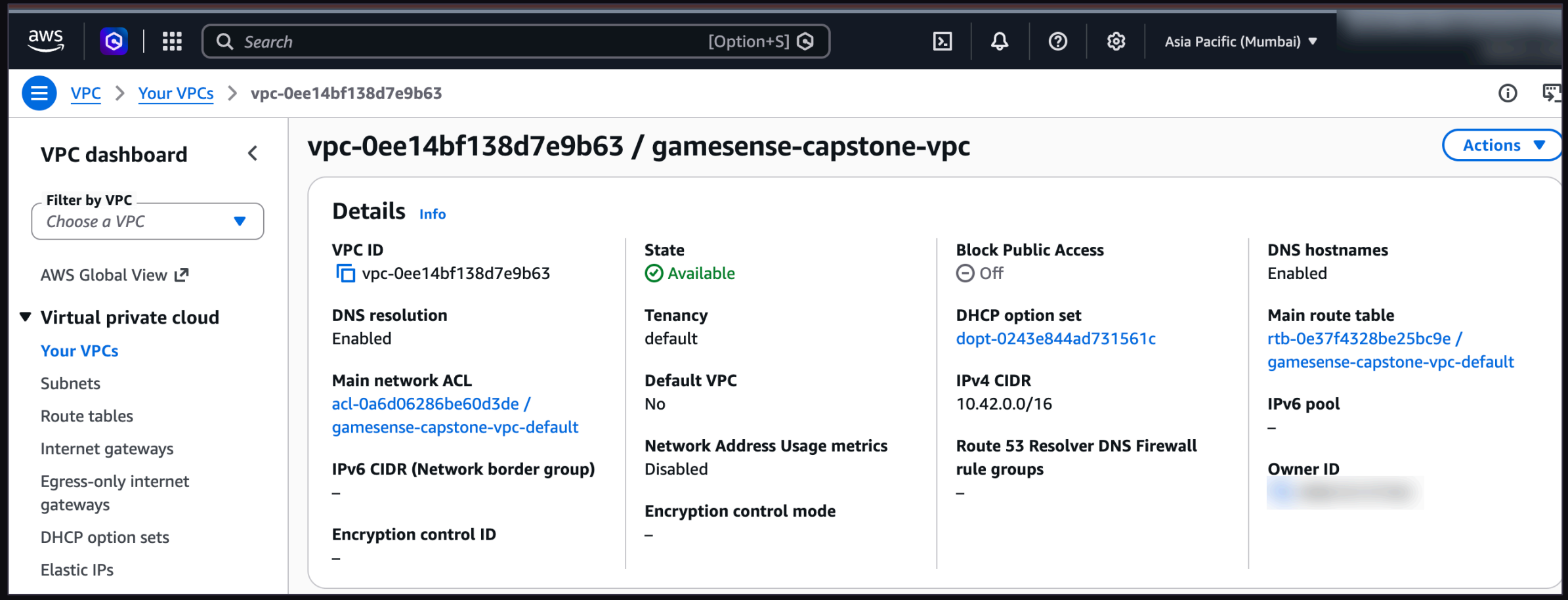
AWS Console: EKS + VPC in ap-south-1 (M7) (1/2)



EKS cluster gamesense-capstone: Active, Kubernetes 1.35

EVIDENCE

AWS Console: EKS + VPC in ap-south-1 (M7) (2/2)



gamesense-capstone-vpc: Available, 10.42.0.0/16

DEPLOYMENT

Kubernetes, Helm & GitOps (M8)

Helm chart

Frontend and backend Deployments with 2 replicas each, readiness and liveness probes, ClusterIP Services, and one Ingress (/ → frontend, /api → backend).

Ingress

Traefik publishes the Ingress through an AWS load balancer, so no DNS purchase is needed for the demo.

Secrets & data

Database URL and passwords live in Kubernetes Secrets created outside Git. The catalog bundle is mounted from a ConfigMap.

Argo CD

Watches main and renders helm/ with values-eks.yaml layered on top. Auto-sync, prune, and self-heal are on.

Push to deploy

A push to main makes CI publish images and commit the new tag; Argo CD syncs and rolls the pods. No kubectl or helm is run by hand.

One owner

Helm installed the release once. Argo CD now owns ongoing changes, so two controllers never fight over it.

EVIDENCE

kubectl get pods (M8)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % kubectl get pods -n gamesense

NAME                                          READY   STATUS    RESTARTS   AGE
gamesense-backend-6bd7c45dc4-ls6td            1/1     Running   0          20m
gamesense-backend-6bd7c45dc4-nccf8            1/1     Running   0          20m
gamesense-frontend-6576475fc7-j4zsp           1/1     Running   0          72m
gamesense-frontend-6576475fc7-w226x           1/1     Running   0          72m
gamesense-monitoring-grafana-b648b68cf-cgggm  1/1     Running   0          74m
gamesense-monitoring-prometheus-798d5ddc6f-5lhqk  1/1     Running   0          74m
gamesense-postgres-7fbc744f9-zfpnm            1/1     Running   0          76m
```

All pods Running: 2 backend and 2 frontend replicas, plus PostgreSQL, Prometheus, and Grafana

EVIDENCE

kubectl get svc (M8)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % kubectl get svc -n gamesense; kubectl get svc -n gamesense-ingress

NAME                              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
gamesense-backend                 ClusterIP   172.20.129.200   <none>        8000/TCP   72m
gamesense-frontend                ClusterIP   172.20.29.202    <none>        8080/TCP   72m
gamesense-monitoring-grafana      ClusterIP   172.20.216.70    <none>        3000/TCP   74m
gamesense-monitoring-prometheus   ClusterIP   172.20.157.199   <none>        9090/TCP   74m
gamesense-postgres                ClusterIP   172.20.31.70     <none>        5432/TCP   76m
NAME                       TYPE           CLUSTER-IP       EXTERNAL-IP                                                                 PORT(S)        AGE
gamesense-ingress-traefik   LoadBalancer   172.20.197.168   adfed29e3216b463b988d447f9b53ee2-1879947908.ap-south-1.elb.amazonaws.com   80:31109/TCP   76m
```

ClusterIP Services for the app, and the Traefik LoadBalancer

EVIDENCE

helm list + Ingress (M8)

```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % helm list -n gamesense; kubectl get ingress -n gamesense

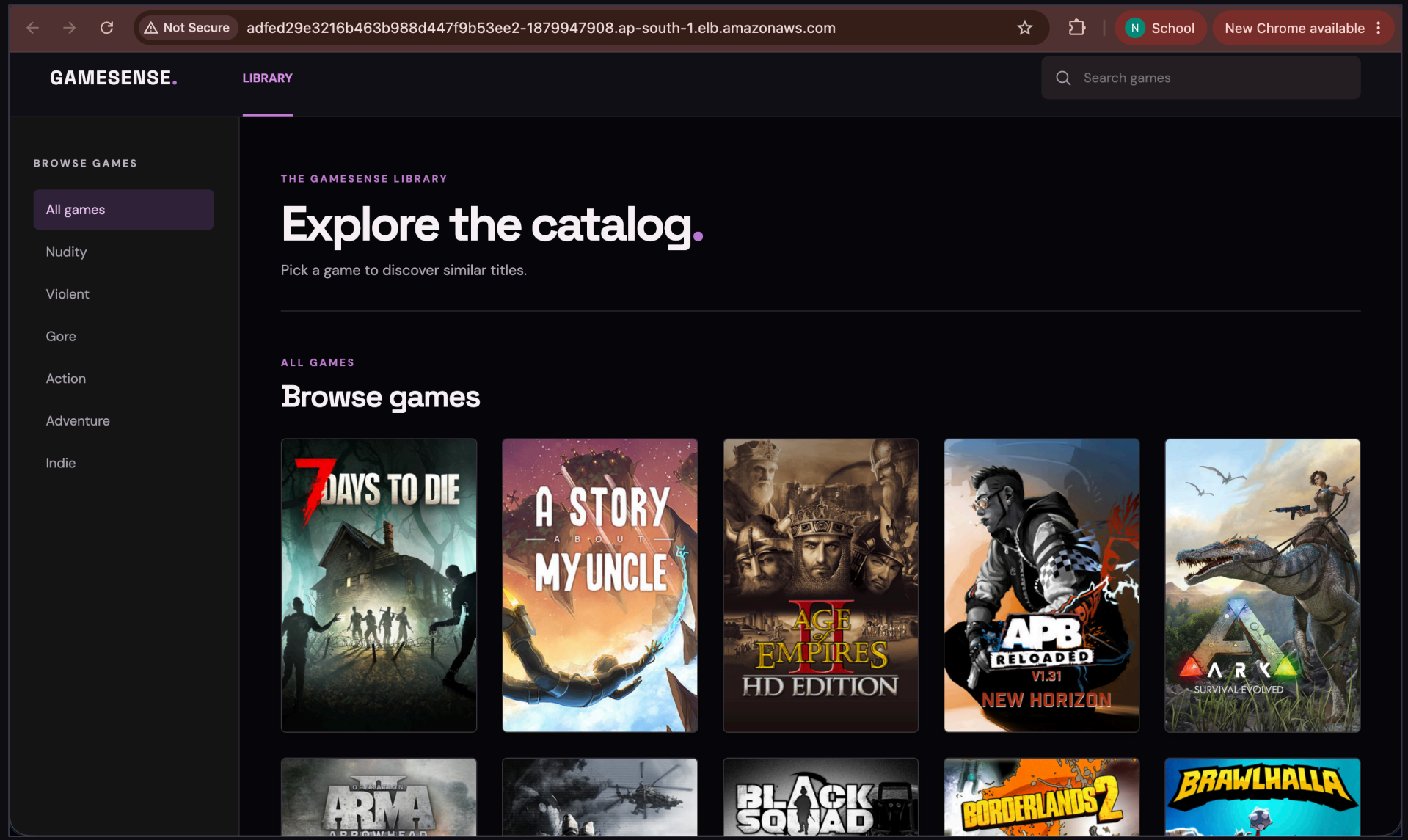
NAME                    NAMESPACE       REVISION        UPDATED                                 STATUS          CHART                           APP VERSION
gamesense               gamesense       3               2026-10-16 21:11:58.200617 +0530 IST    deployed        gamesense-0.2.0                 0.2.0
gamesense-monitoring    gamesense       1               2026-10-16 20:17:56.278252 +0530 IST    deployed        gamesense-monitoring-0.1.0

NAME        CLASS                       HOSTS   ADDRESS                                                                 PORTS   AGE
gamesense   gamesense-ingress-traefik   *       adfed29e3216b463b988d447f9b53ee2-1879947908.ap-south-1.elb.amazonaws.com   80      72m
```

Helm releases deployed; the Ingress routes / and /api through the load balancer

EVIDENCE

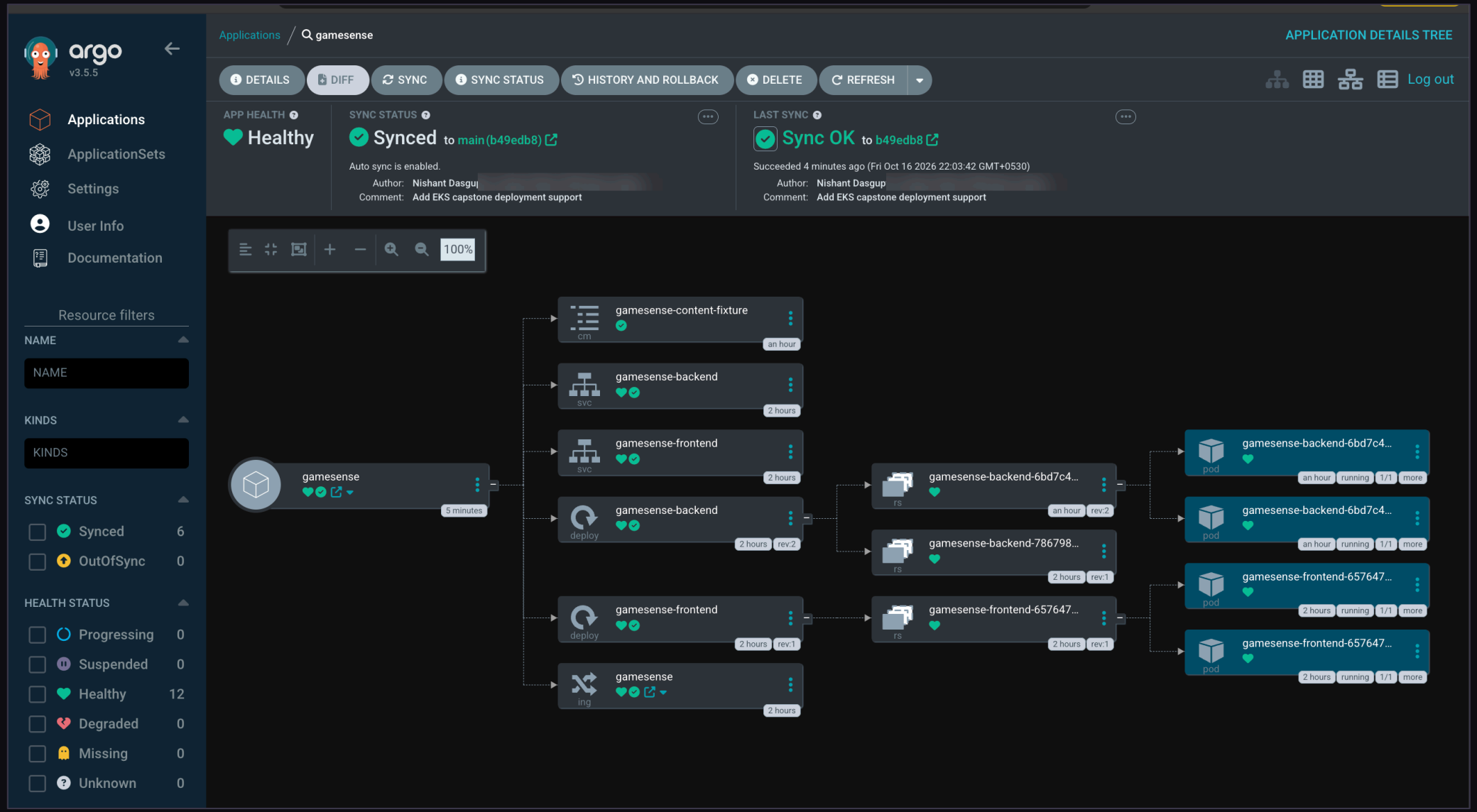
App live through the Ingress (M8)



GameSense on EKS at the AWS load balancer hostname

EVIDENCE

Argo CD: synced and healthy (GitOps)



Tracks main; auto-sync and self-heal on; full resource tree healthy

PROMETHEUS · GRAFANA

Observability (M9)

Instrumented API

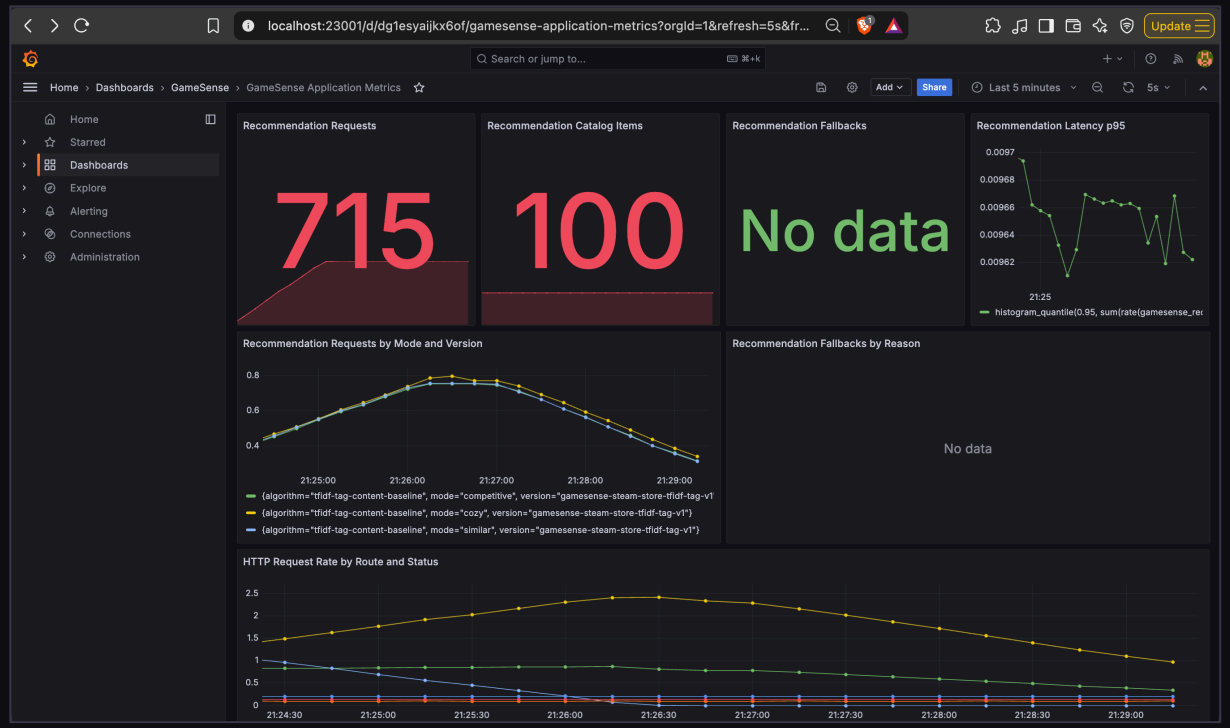
/metrics exposes request counts, latency histograms, and recommender metrics labelled by mode, algorithm, version, and data snapshot.

Prometheus

Discovers backend pods by annotation, scoped to the app namespace. Both replicas show as UP.

Grafana

Provisioned datasource and dashboard: requests, catalog size, fallbacks, p95 latency, and HTTP rate by route.



Grafana on EKS: 715 recommendation requests, 100-game catalog

EVIDENCE

Backend /metrics (M9)

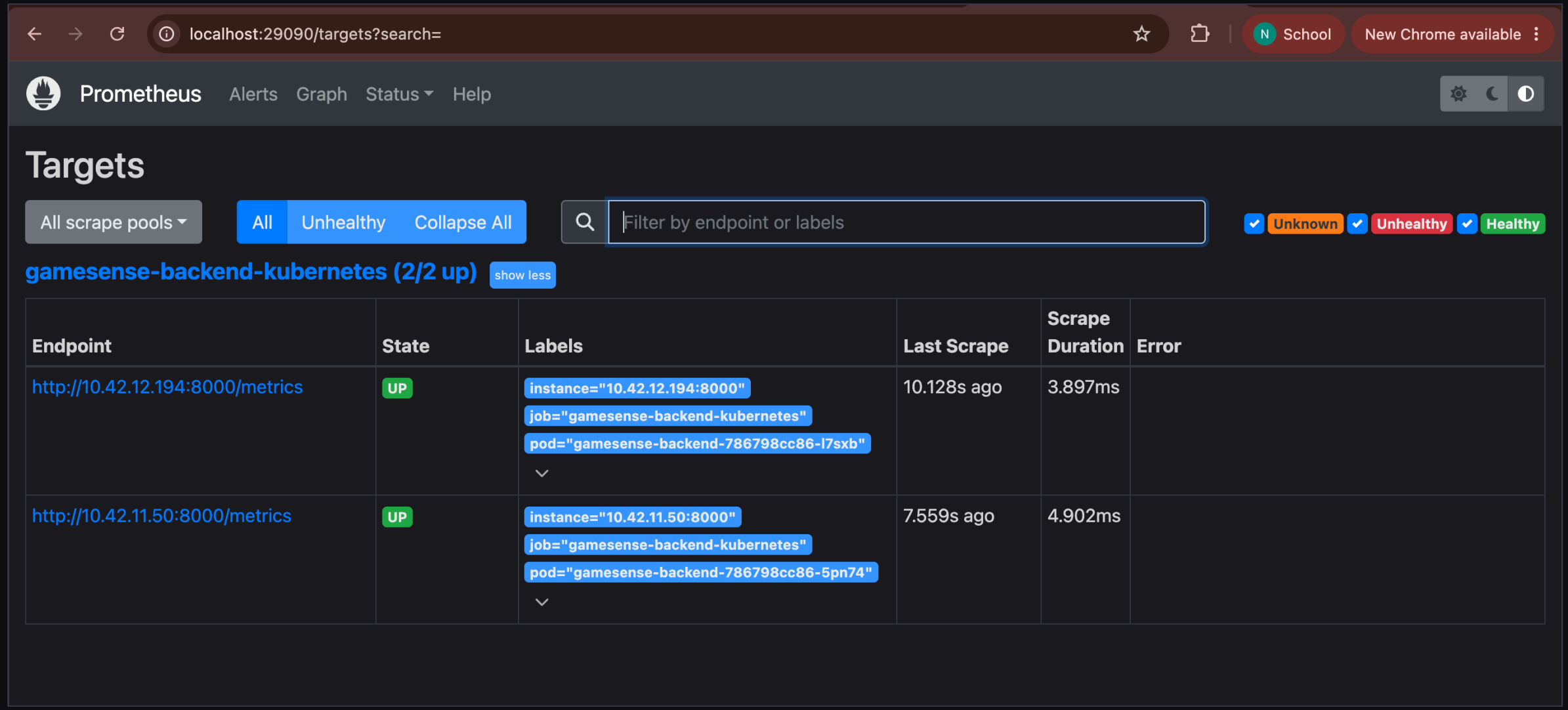
```
nishantdasgupta@Nishants-M4-MacBook-Pro GameSense % kubectl -n gamesense exec deploy/gamesense-backend -- <GET http://localhost:8000/metrics>

gamesense_http_requests_total{method="GET",route="/metrics",status_code="200"} 85.0
gamesense_http_requests_total{method="GET",route="/ready",status_code="200"} 124.0
gamesense_http_requests_total{method="GET",route="/health",status_code="200"} 62.0
gamesense_http_requests_total{method="GET",route="/api/games",status_code="200"} 378.0
gamesense_http_requests_total{method="POST",route="/api/recommendations",status_code="422"} 165.0
gamesense_http_requests_total{method="POST",route="/api/recommendations",status_code="200"} 253.0
gamesense_http_request_duration_seconds_count{method="GET",route="/metrics"} 85.0
gamesense_http_request_duration_seconds_sum{method="GET",route="/metrics"} 0.18358526800147956
gamesense_http_request_duration_seconds_count{method="GET",route="/ready"} 124.0
gamesense_http_request_duration_seconds_sum{method="GET",route="/ready"} 0.29342397800428444
gamesense_http_request_duration_seconds_count{method="GET",route="/health"} 62.0
gamesense_http_request_duration_seconds_sum{method="GET",route="/health"} 0.045530123001299216
gamesense_http_request_duration_seconds_count{method="GET",route="/api/games"} 378.0
gamesense_http_request_duration_seconds_sum{method="GET",route="/api/games"} 0.7152589260090281
gamesense_http_request_duration_seconds_count{method="POST",route="/api/recommendations"} 418.0
gamesense_http_request_duration_seconds_sum{method="POST",route="/api/recommendations"} 1.596370986004331
gamesense_recommendation_seconds_count 253.0
gamesense_recommendation_seconds_sum 0.9747134040003402
gamesense_recommendation_requests_total{algorithm="tfidf-tag-content-baseline",data_snapshot="kaggle.steam-store-games-clean-local-
snapshot",mode="cozy",version="gamesense-steam-store-tfidf-tag-v1"} 223.0
gamesense_recommendation_requests_total{algorithm="tfidf-tag-content-baseline",data_snapshot="kaggle.steam-store-games-clean-local-
snapshot",mode="similar",version="gamesense-steam-store-tfidf-tag-v1"} 15.0
gamesense_recommendation_requests_total{algorithm="tfidf-tag-content-baseline",data_snapshot="kaggle.steam-store-games-clean-local-
snapshot",mode="competitive",version="gamesense-steam-store-tfidf-tag-v1"} 15.0
gamesense_recommendation_model_info{algorithm="tfidf-tag-content-baseline",data_snapshot="kaggle.steam-store-games-clean-local-snapshot",version="gamesense-steam-store-
tfidf-tag-v1"} 1.0
gamesense_recommendation_catalog_items{algorithm="tfidf-tag-content-baseline",data_snapshot="kaggle.steam-store-games-clean-local-snapshot"} 100.0
gamesense_demo_catalog_games 100.0
```

Prometheus-format metrics from a backend pod on EKS

EVIDENCE

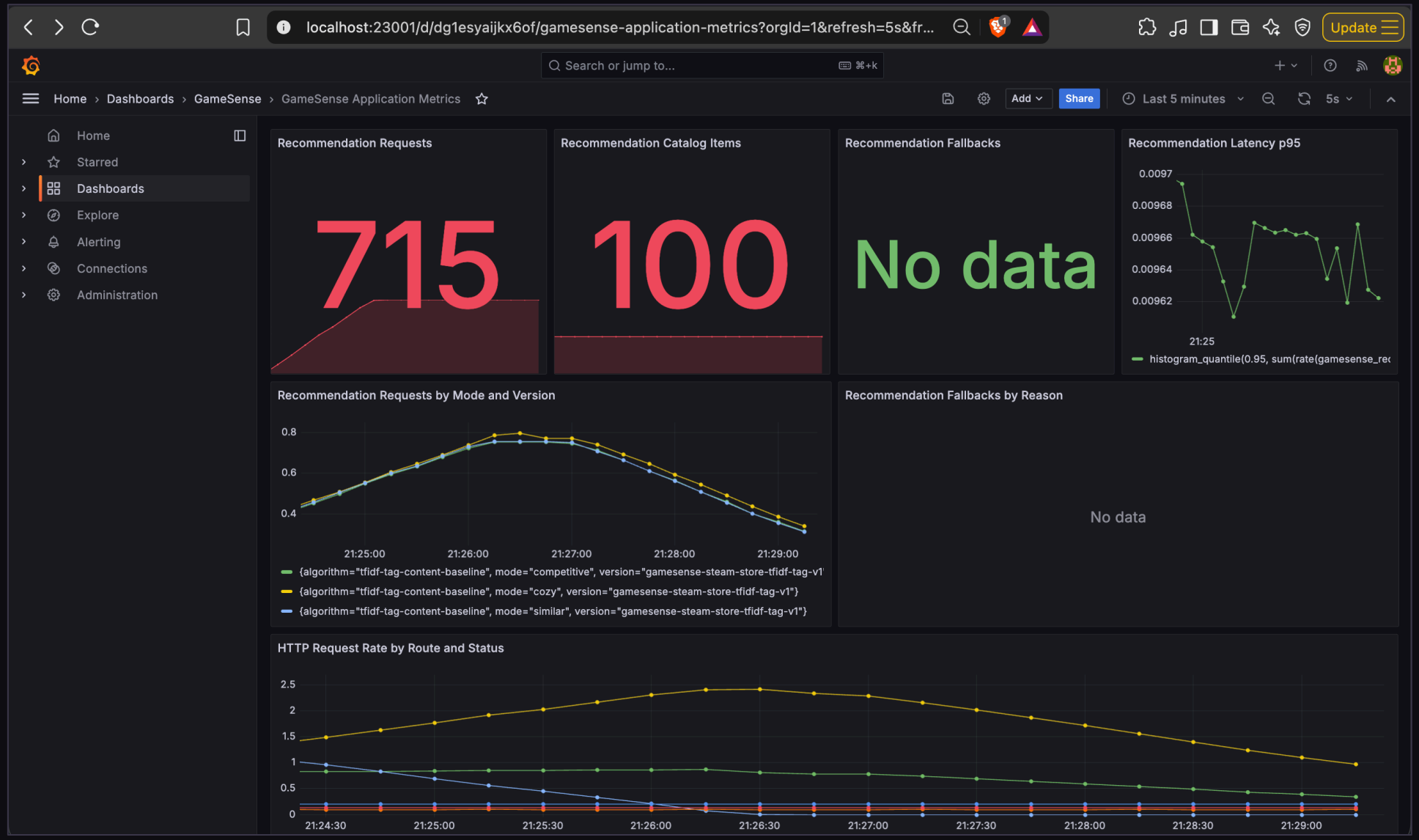
Prometheus targets UP (M9)



gamesense-backend-kubernetes: 2/2 targets UP

EVIDENCE

Grafana dashboard (M9)



Live recommendation requests, catalog size, latency, and HTTP rate by route

WHY IT IS BUILT THIS WAY

Design decisions & trade-offs

Traefik instead of the AWS ALB controller
Fewer IAM moving parts for a short-lived demo. The chart still supports ALB via ingress.className.

Disposable PostgreSQL (emptyDir)
Good enough for a demo; production would use RDS or a persistent volume.

GitOps image tags
CI writes the deployed tag to Git, so every deployment is a reviewable commit.

Data licensing
Raw Kaggle data and model bundles are never committed. The cloud demo serves only a 100-game subset.

Secrets out of Git
Tokens and passwords live in Kubernetes Secrets. Trivy secret scan and history checks are clean.

Cost control
Short-lived cluster, one NAT gateway, two small nodes, and terraform destroy straight after the demo.

WHAT I WOULD KEEP AND CHANGE

Key learnings

Small configuration details break deployments
The Ingress returned 404 until its class matched the name Traefik actually registered. Check the cluster's real resource names, not the defaults.

GitOps needs one source of truth
Deployed image tags had to move into Git, written by CI, before Argo CD could be the only deployer.

Helm merges maps
An empty annotations map does not remove a default annotation; it takes an explicit null.

Security gates belong before publish
Trivy runs before anything reaches GHCR, so a vulnerable image never becomes deployable.

Data has licences too
The dataset's terms shaped what could be committed or shipped to the cloud.

Next steps
TLS and DNS, RDS instead of emptyDir, an ALB controller with IRSA, and alerting rules in Prometheus.

WHAT THE DEMO SHOWS

Live walkthrough

1. Codebase tour: folder structure and key files
2. Push a visible change → GitHub Actions runs end-to-end
3. Terraform plan / apply output + AWS Console (VPC, EKS)
4. Argo CD picks up the tag commit and syncs
5. kubectl get pods, svc, ingress in the cluster
6. The updated app live through the load balancer
7. Prometheus Targets UP · Grafana live dashboard

Thank you

github.com/NDGCodes/GameSense

Rubric evidence: screenshots and logs submitted with this deck.